

DOCUMENTACIÓN PROYECTO M1 DAVID SANTIAGO RUBIO MONTERO

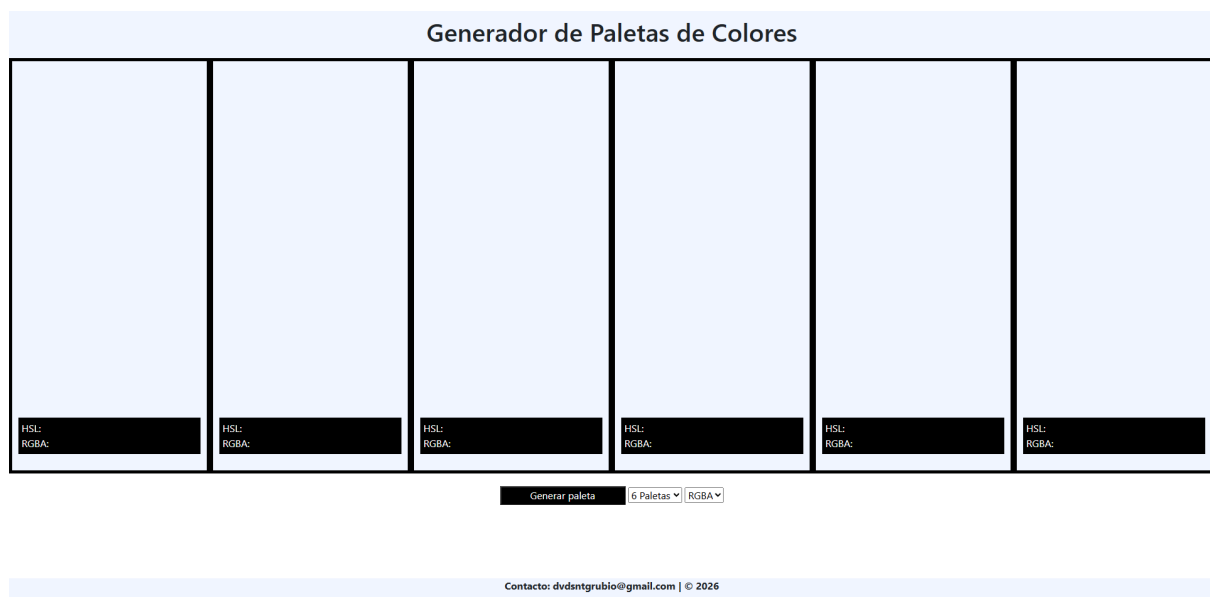
En este documento encontrarás el flujo principal de la WEB y el uso documentado de IA para la realización del proyecto

Flujo Principal

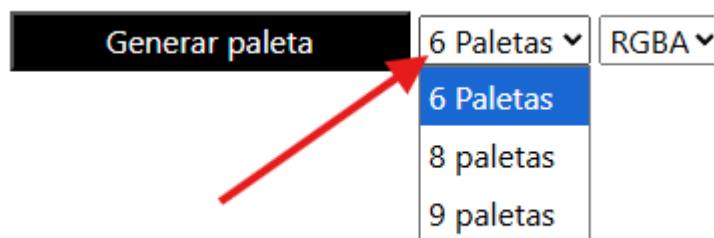
El flujo de la WEB está diseñado para ser intuitivo, permitiendo al usuario pasar de la visualización a la personalización en tres pasos sencillos:

1. Configuración del Espacio de Trabajo

Al cargar la WEB, el usuario se encuentra con una interfaz limpia que muestra por defecto **6 paletas** en formato **RGBA**.



- **Selección de cantidad:** El usuario puede interactuar con el menú desplegable para expandir su lienzo a **8 o 9 paletas**.



-

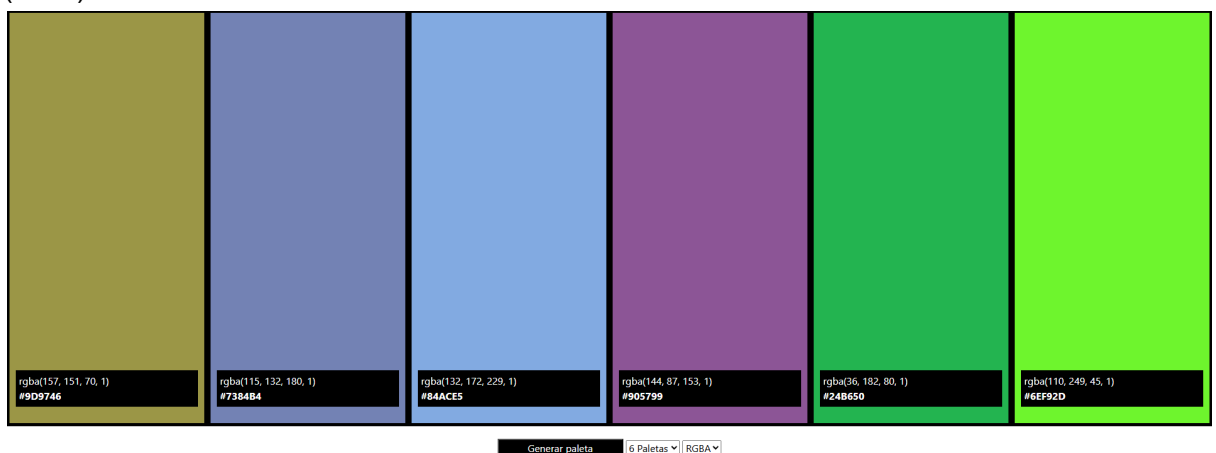
- **Respuesta dinámica:** El DOM se ajusta automáticamente, creando o eliminando contenedores `div` en tiempo real mediante un bucle `for` y manteniendo la simetría visual.

2. Generación de Contenido Creativo

Una vez definida la cantidad de muestras, el usuario acciona el botón "**Generar paleta**".



- **Randomización:** El motor de la app genera valores aleatorios de Rojo, Verde y Azul (RGB).



- **Feedback visual:** El texto del botón cambia temporalmente a "**Paleta generada!!!!**" para confirmar la acción, mientras los fondos de las cajas se actualizan con los nuevos colores.

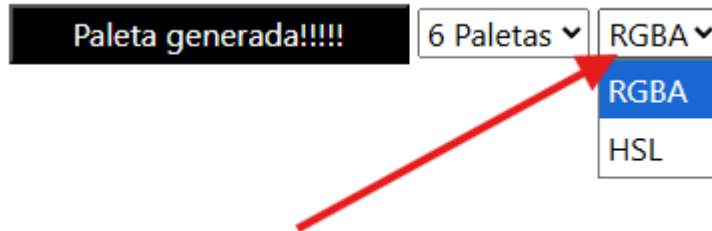


- **Persistencia:** Los valores numéricos del color se almacenan de forma invisible en el atributo `dataset` de cada caja, permitiendo futuras conversiones sin perder el tono actual.

3. Personalización y Exportación de Datos

Con la paleta generada, el usuario puede refinar cómo desea visualizar la información técnica:

- **Cambio de Formato:** A través del selector de formatos, el usuario puede alternar entre **RGBA** y **HSL**.



-
- **Traducción Instantánea:** La app utiliza algoritmos de conversión para transformar los valores almacenados, actualizando los párrafos informativos sin necesidad de refrescar la página o cambiar el color de fondo.
- **Visualización Hexadecimal:** Independientemente del formato elegido, el código **HEX** se muestra siempre resaltado en negrita como valor de referencia estándar para diseño web.

rgba(136, 166, 156, 1) #8A69C	rgba(84, 207, 240, 1) #54CFF0	rgba(6, 106, 63, 1) #066A3F	rgba(102, 250, 46, 1) #66FA2E	rgba(21, 194, 162, 1) #15C2A2	rgba(175, 88, 106, 1) #AF586A
---	---	---	---	---	---

DOCUMENTACIÓN IA EN EL PROYECTO

Primer Prompt

estoy modificando mis clases caja-color esas clase son un div, ese div tiene adentro una etiqueta p la etiqueta p se puede modificar pero cuando le doy estilos al div no deja por que sucede esto

Respuesta IA



El problema principal es una confusión entre cómo llamas a los elementos en tu **HTML** y cómo los escribes en tu **CSS**.

En tu CSS estás usando un **ID** (`#caja-color`), pero en tu pregunta dices que estás modificando tus **clases** (`.caja-color`). En el mundo del desarrollo web, esto es como intentar abrir una puerta con la llave equivocada.

1. ¿Clase o ID?

- Si en tu HTML usaste `class="caja-color"`: Debes usar un punto en CSS: `.caja-color`.
- Si en tu HTML usaste `id="caja-color"`: Debes usar el numeral: `#caja-color`.

Como mencionaste que son "clases" (en plural), lo más probable es que debas cambiar el `#` por un `.` en tu código.

2. El tamaño del Div

Si después de cambiar el punto el fondo sigue sin verse, puede ser por el **contenido**.

Un `div` es como una caja vacía; si el párrafo (`p`) que tiene adentro tiene sus propios estilos (como el `background-color: antiquewhite` que pusiste), el párrafo "tapará" el color del div.

Además, si el `div` no tiene contenido o el contenido tiene dimensiones extrañas, el `div` podría tener una altura de **0 píxeles**, lo que lo hace invisible.

segundo prompt

Quiero darle un alto a mis clases `caja.color` que ocupa un 80 o 90% del `main` que es la caja que contiene a mis `divs` pero cuando especifíco su altura en % no cambia la vista en la pantalla o se ve mas pequeño como puedo hacer lo que quiero?

Este es uno de los problemas más comunes en CSS. La razón por la que el porcentaje (%) no te funciona como esperas es porque, en CSS, **un porcentaje de altura solo funciona si el padre tiene una altura definida explícitamente**.

En tu código, el `main` tiene `flex: 1`, lo que lo hace crecer para llenar el espacio, pero no tiene una "altura fija" numérica que sus hijos (las `.caja-color`) puedan usar para calcular ese 80% o 90%.

¿Cómo solucionarlo?

Tienes dos opciones principales para lograr que tus cajas ocupen casi todo el `main`:

1. La solución con unidades de medida modernas (vh)

En lugar de usar %, puedes usar `vh` (viewport height). Esto le dice a la caja que mida un porcentaje de la **pantalla completa**, sin importar lo que mida el padre.

Como tienes un `header` y un `footer`, podrías usar algo como esto:

CSS



```
.caja-color {  
  background-color: rgb(222, 184, 184);  
  width: 100%;  
  height: 80vh; /* Ocupa el 80% del alto de la ventana del navegador */  
  border: 5px solid black;  
  display: flex;  
  align-items: flex-end;  
}
```

Tercer prompt

Tengo una situación estoy usando flexbox para manejar mis divs y mi main, lo que pasa es que agregue un botón en el main y el flexbox lo está modificando pero yo quiero que el botón se alinee por debajo de los divs como si fuera una columna se que se podría hacer con grid pero hay alguna manera de hacerlo con flexbox

¡Claro que se puede con Flexbox! El problema es que ahora mismo tienes tu `main` configurado con `flex-direction: row`, lo que obliga a que todo lo que metas dentro (tus 6 cajas y el botón) se ponga uno al lado del otro.

Para solucionar esto sin romper el diseño de tus cajas de colores, necesitas **agrupar**.

La solución: Crear un "Contenedor de Cajas"

La mejor forma de hacerlo con Flexbox es envolver tus 6 cajas de colores en un `div` propio. Así, el `main` solo tendrá dos hijos directos: el grupo de cajas y el botón.

Aquí tienes los pasos:

1. Ajusta tu HTML:

Envuelve las cajas en un nuevo contenedor (por ejemplo, con la clase `.contenedor-cajas`).

HTML



```
<main>
  <div class="contenedor-cajas">
    </div>

  <button id="boton">Generar Paleta</button>
</main>
```

cuarto prompt

Estoy intentado seleccionar los `<option>` de mi `<select>` con JS para cuando yo le de click a alguno me genere un `console.log`, en teoría ya debería esta funcionando

pero no ocurre lo que quiero, por que pasa esto?

1. El evento correcto no es `click`

En un elemento `<select>`, el usuario no interactúa haciendo "click" en cada opción de forma individual para que JS lo detecte así. El evento estándar para los menús desplegables es `change`. Este evento se dispara cada vez que la selección del menú cambia.

2. No se seleccionan las opciones por separado

Cuando haces `document.getElementById('boton-seleccionador')[0]`, estás intentando acceder al elemento como si fuera un array, pero `getElementById` solo devuelve **un único objeto** (el select completo).

La forma correcta de hacerlo

En lugar de ponerle un "escuchador" a cada opción, se le pone **uno solo al** `<select>` y luego le preguntas qué valor tiene seleccionado.

Aquí te explico la lógica:

1. Seleccionas el elemento padre (el `select`).
2. Le agregas el evento `change`.
3. Dentro de la función, usas `this.value` para saber qué opción eligió el usuario.

JavaScript



```
const seleccionador = document.getElementById('boton-seleccionador');

seleccionador.addEventListener('change', function() {
  // 'this.value' toma el valor del atributo 'value' del option seleccionado
  if (this.value === "6-colores") {
    console.log("Elegiste 6 colores");
  } else if (this.value === "8-colores") {
    console.log("Elegiste 8 colores");
  }
});
```

quinto prompt

contexto:

archivo script.js

Estoy usando JS para agregar nuevos elementos a mi contenedor en html.

objetivo:

quiero saber como remover esos elementos del html que posteriormente agregue instrucciones

retorna una explicación y el código

Entiendo perfectamente el punto en el que estás. El problema con tu código actual es que estás intentando remover un elemento que acabas de crear en memoria (`let nueva_paleta`), pero que aún no existe realmente dentro del HTML, o intentas removerlo usando una referencia que se pierde cada vez que la función termina.

Para remover elementos que agregaste dinámicamente, tienes dos estrategias principales:

1. Remover el último elemento agregado

Si quieres que al seleccionar una opción se borre la última caja que se puso, puedes usar

`lastElementChild`.

JavaScript



```
// Seleccionamos el último hijo que tenga el contenedor y lo eliminamos
if (contenedor_cajas.lastElementChild) {
  contenedor_cajas.removeChild(contenedor_cajas.lastElementChild);
}
```

2. Limpiar todo el contenedor (Reiniciar)

Si tu objetivo es que al cambiar de "6" a "8" colores, se borren los anteriores para poner los nuevos, lo más sencillo es vaciar el contenedor antes de empezar a agregar.

JavaScript



```
// Esto borra todo el contenido HTML dentro del contenedor
contenedor_cajas.innerHTML = '';
```

sexto prompt

estoy intentando que el option identificado con el value 9-colores agregue los elementos indicados en el for, el contador siempre estará iniciado en 6 o en en 8 contenedores div la idea es que cuando se seleccione este, los contenedores se agreguen a hasta 9



Este es un error muy común al empezar con formularios en JavaScript. Hay dos razones principales por las que tu código no está funcionando como esperas:

1. El evento correcto no es `click`

En un elemento `<select>`, el usuario no interactúa haciendo "click" en cada opción de forma individual para que JS lo detecte así. El evento estándar para los menús desplegables es `change`. Este evento se dispara cada vez que la selección del menú cambia.

2. No se seleccionan las opciones por separado

Cuando haces `document.getElementById('boton-seleccionador')[0]`, estás intentando acceder al elemento como si fuera un array, pero `getElementById` solo devuelve **un único objeto** (el select completo).

La forma correcta de hacerlo

En lugar de ponerle un "escuchador" a cada opción, se le pone **uno solo al** `<select>` y luego le preguntas qué valor tiene seleccionado.

Aquí te explico la lógica:

1. Seleccionas el elemento padre (el `select`).
2. Le agregas el evento `change`.
3. Dentro de la función, usas `this.value` para saber qué opción eligió el usuario.

séptimo prompt

Estoy intentando asignar un color random a los contenedores hijos a través de su clase pero al visualizar el html no da el resultado que quiero como se puede solucionar [script.js](#)

1. El error de nombres y lógica en la función

En tu código llamaste a la variable igual que a la función: `color_random = Math.floor(...)`. Esto borra la función después de la primera ejecución. Además, el array `aleatorio_rgba` debe vaciarse en cada vuelta del bucle para que cada caja tenga un color distinto.

2. Cómo recorrer y aplicar el estilo

`paletas` es una lista (HTMLCollection). No puedes hacer `paletas.style`. Debes usar el índice `[i]` para decirle a JavaScript: "Aplica este color a la caja número i".

Código corregido y optimizado

JavaScript



```
// 1. Función para generar UN color RGBA aleatorio
function generarColor() {
  let r = Math.floor(Math.random() * 256);
  let g = Math.floor(Math.random() * 256);
  let b = Math.floor(Math.random() * 256);
  let a = Math.random().toFixed(2);
  return `rgba(${r}, ${g}, ${b}, ${a})`;
}

// 2. Seleccionamos todas las cajas por su clase
const paletas = document.getElementsByClassName('caja-color');

// 3. Recorremos la colección de cajas
for (let i = 0; i < paletas.length; i++) {
  // Generamos un color nuevo para cada vuelta del bucle
  let nuevoColor = generarColor();

  // Aplicamos el color a la paleta actual [i]
  paletas[i].style.backgroundColor = nuevoColor;

  // Opcional: Mostrar el código en el texto del párrafo
  let parrafo = paletas[i].querySelector('p');
  if (parrafo) {
    parrafo.innerText = `Color: ${nuevoColor}`;
  }
}
```

octavo prompt

Contexto: Estoy usando JS para realizar una paleta random de colores, ya tengo implementada la lógica que se va a usar para randomizar los colores

Objetivo:

se quiere convertir los colores de formato RGBA a formato HEX y imprimirlos en pantalla con la clase de estilos correspondiente al darle click al boton principal

INSTRUCCIONES:

1. crear una función que transforme el formato RGBA (el cual su lógica ya genera colores random) al formato HSL
 2. imprimir en pantalla el código HSL O RGBA (dependiendo de cuál formato se escogió por el usuario) junto con su código HEX
 3. los resultados se tiene que visualizar cuando se le dé al botón generar paleta a continuación te voy a mostrar el código actual e imaginas del front para que tengas una idea de como esta actualmente el trabajo
- Respuesta:

1. Funciones de Conversión

Primero, añadimos las "traductoras" que transformarán los números base a los formatos que necesitas.

JavaScript

```
// Convierte R, G, B a Hexadecimal
function rgbToHex(r, g, b) {
    return "#" + [r, g, b].map(x => x.toString(16).padStart(2, '0')).join('')
}

// Convierte R, G, B a HSL
function rgbToHsl(r, g, b) {
    r /= 255; g /= 255; b /= 255;
    const max = Math.max(r, g, b), min = Math.min(r, g, b);
    let h, s, l = (max + min) / 2;

    if (max === min) {
        h = s = 0;
    } else {
        const d = max - min;
        s = l > 0.5 ? d / (2 - max - min) : d / (max + min);
        switch (max) {
            case r: h = (g - b) / d + (g < b ? 6 : 0); break;
            case g: h = (b - r) / d + 2; break;
            case b: h = (r - g) / d + 4; break;
        }
        h /= 6;
    }
    return `hsl(${Math.round(h * 360)}, ${Math.round(s * 100)}%, ${Math.round(l * 100)}%)`
}
```

2. Actualización de la lógica del Botón

Modificaremos tu evento de clic para que guarde los valores en la caja (usando `dataset`) y muestre ambos códigos.

JavaScript

```
boton.addEventListener('click', function() {
  const textoOriginal = this.innerText;
  this.innerText = "Paleta generada!!!!";

  for (let i = 0; i < paletas.length; i++) {
    // Generamos números base
    let r = Math.floor(Math.random() * 256);
    let g = Math.floor(Math.random() * 256);
    let b = Math.floor(Math.random() * 256);

    // Guardamos los valores en la caja para que el selector de formato pueda
    paletas[i].dataset.r = r;
    paletas[i].dataset.g = g;
    paletas[i].dataset.b = b;

    // Pintamos la caja y los textos
    actualizarInterfazCaja(paletas[i]);
  }

  setTimeout(() => {
    this.innerText = textoOriginal;
  }, 500);
});
```

Esta función se encarga de leer qué formato quiere el usuario y escribirlo en el párrafo junto al HEX.

JavaScript

```
function actualizarInterfazCaja(caja) {
  const r = parseInt(caja.dataset.r);
  const g = parseInt(caja.dataset.g);
  const b = parseInt(caja.dataset.b);
  const formatoSeleccionado = formatos.value; // El select de RGBA/HSL

  let textoFormato = "";
  let hex = rgbToHex(r, g, b);

  // Decidimos qué mostrar arriba
  if (formatoSeleccionado === "btn-hsl") {
    textoFormato = rgbToHsl(r, g, b);
  } else {
    textoFormato = `rgba(${r}, ${g}, ${b}, 1)`;
  }

  // Aplicamos el color de fondo (siempre usamos el formato seleccionado para el color)
  caja.style.backgroundColor = textoFormato;

  // Imprimimos en el párrafo
  let parrafo = caja.querySelector('p');
  if (parrafo) {
    parrafo.classList.add('textcolor');
    // Usamos innerHTML para el salto de línea y la negrita en HEX
    parrafo.innerHTML = `${textoFormato}<br><strong>${hex}</strong>`;
  }
}

// LOGICA DE SELECCION DE FORMATOS
// Cuando el usuario cambie de RGBA a HSL, actualizamos todas las cajas sin cambiar el color
formatos.addEventListener('change', function() {
  for (let i = 0; i < paletas.length; i++) {
    if (paletas[i].dataset.r) {
      actualizarInterfazCaja(paletas[i]);
    }
  }
});
```